VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

D MURALI SATYA SUHAS (1BM24CS086)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by D MURALI SATYA SUHAS **(1BM24CS086)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Mayanka Gupta
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

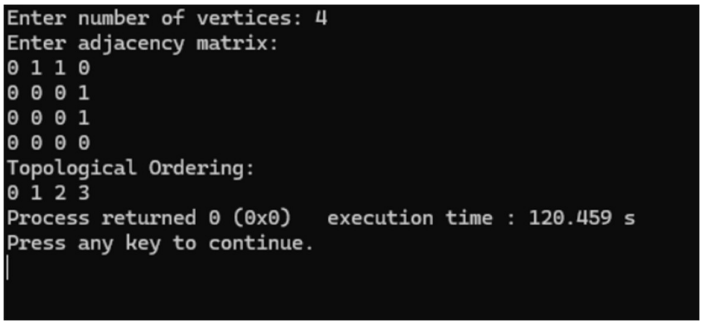
CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

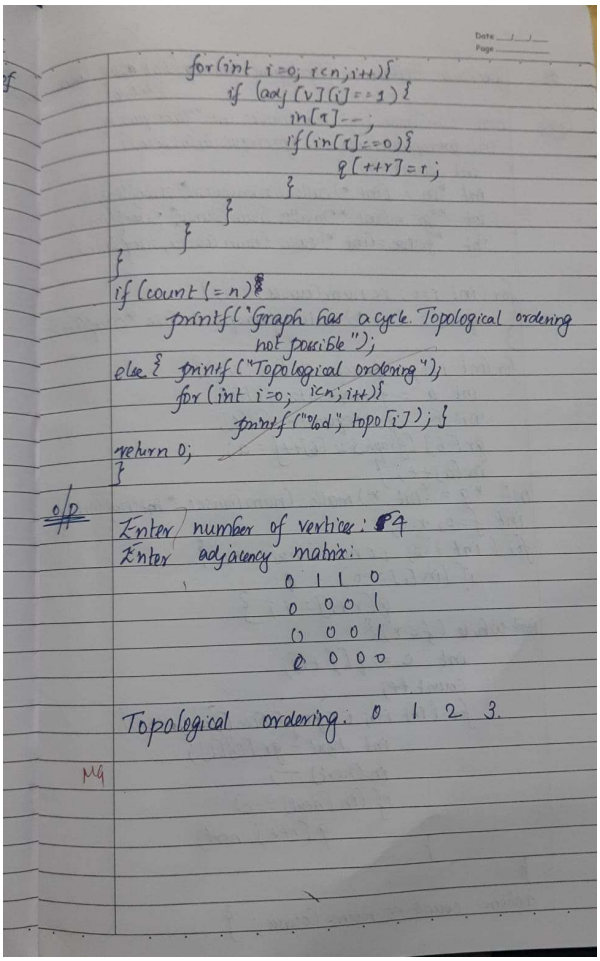
Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int main() {
    int n, adj[MAX][MAX];
    int indegree[MAX] = {0};
    int queue[MAX], front = 0, rear = -1;
    int topo[MAX], count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
    for(int i = 0; i < n; i++) {
        if(indegree[i] == 0) {
            queue[++rear] = i;
        }
```

```
    }
    while(front <= rear) {
        int v = queue[front++];
        topo[count++] = v;
        for(int i = 0; i < n; i++) {
            if(adj[v][i] == 1) {
                indegree[i]--;
                if(indegree[i] == 0) {
                    queue[++rear] = i;
                }
            }
        }
    }
    if(count != n) {
        printf("Graph has a cycle. Topological ordering not possible.\n");
    } else {
        printf("Topological Ordering:\n");
        for(int i = 0; i < n; i++) {
            printf("%d ", topo[i]); }
    }
    return 0;
}
```

Output:





```
for(int i=0; i<n; i++){
    if (adj[v][i]==1){
        in[i]--;
        if(in[i]==0){
            q[++r]=i;
        }
    }
}
}
if (count!=n){
    printf("Graph has a cycle. Topological ordering
    not possible");
else{ printf("Topological ordering");
    for(int i=0; i<n; i++){
        printf("%d", topo[i]); }
return 0;
}
```

o/p

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological ordering: 0 1 2 3.
```

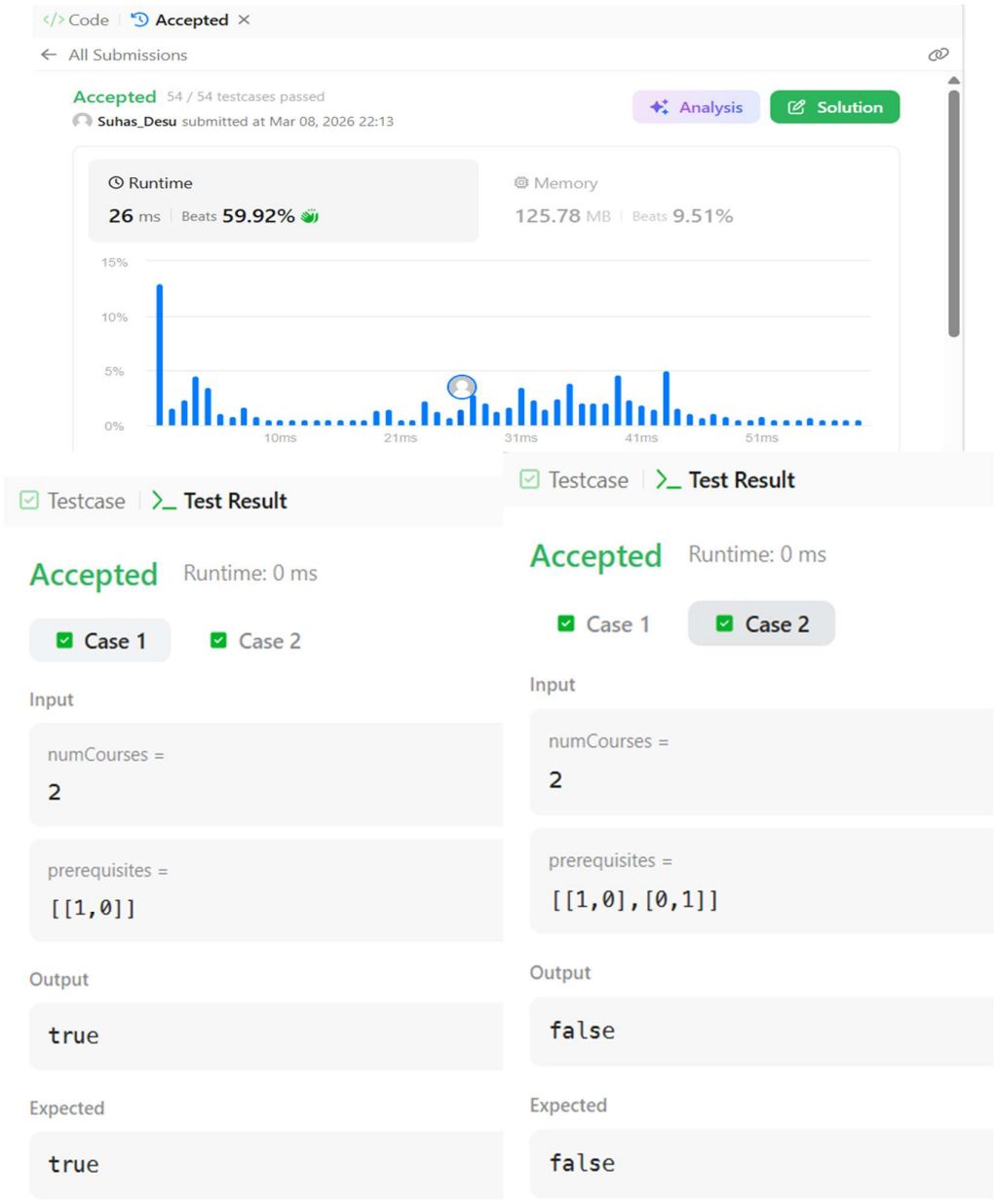
Leet Code exercises on topological sorting.

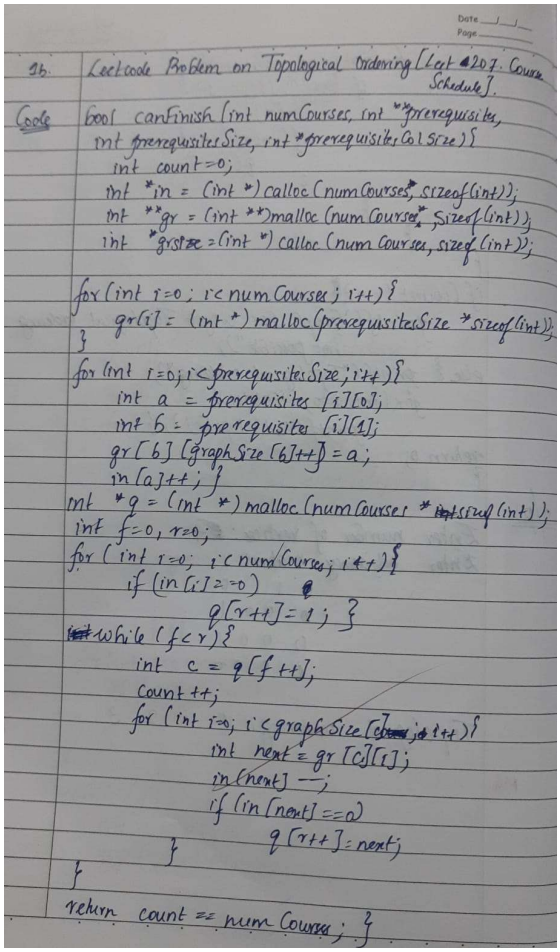
Course Schedule (Leet Code 207):

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int *indegree = (int*)calloc(numCourses, sizeof(int));
    int **graph = (int**)malloc(numCourses * sizeof(int*));
    int *graphSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][graphSize[b]++] = a;
        indegree[a]++;
    }
    int *queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0)
            queue[rear++] = i;
    }
    int count = 0;
    while (front < rear) {
        int course = queue[front++];
        count++;
        for (int i = 0; i < graphSize[course]; i++) {
            int next = graph[course][i];
            indegree[next]--;
            if (indegree[next] == 0)
```

```
            queue[rear++] = next;
        }
    }
    return count == numCourses;
}
```

Output:





Date __/__/__
Page

9b. Leetcode Problem on Topological ordering [Lat 207. Course Schedule].

Code

```
bool canFinish(int numCourses, int **prerequisites,
int prerequisitesSize, int *prerequisitesColSize){
    int count=0;
    int *in = (int *)calloc(numCourses, sizeof(int));
    int **gr = (int **)malloc(numCourses * sizeof(int));
    int *grsize = (int *)calloc(numCourses, sizeof(int));

for(int i=0; i<numCourses; i++){
    gr[i] = (int *)malloc(prerequisitesSize * sizeof(int));
}
for(int i=0;i<prerequisitesSize;i++){
    int a = prerequisites[i][0];
    int b = prerequisites[i][1];
    gr[b][grapSize[b]++] = a;
    in[a]++;}
int *q = (int *)malloc(numCourses * sizeof(int));
int f=0, r=0;
for(int i=0; i<numCourses; i++){
    if(in[i]==0)
        q[r++]=i;}
while(f<r){
    int c = q[f++];
    count++;
    for(int i=0; i<graphSize[c]; i++){
        int next = gr[c][i];
        in[next]--;
        if(in[next]==0)
            q[r++]=next;
    }
}
return count==numCourses;}
```

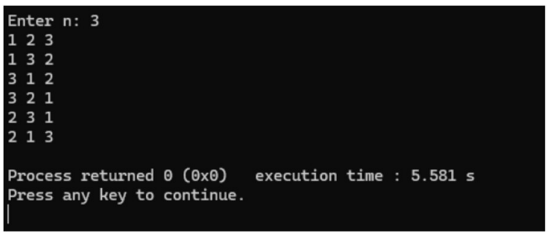
Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

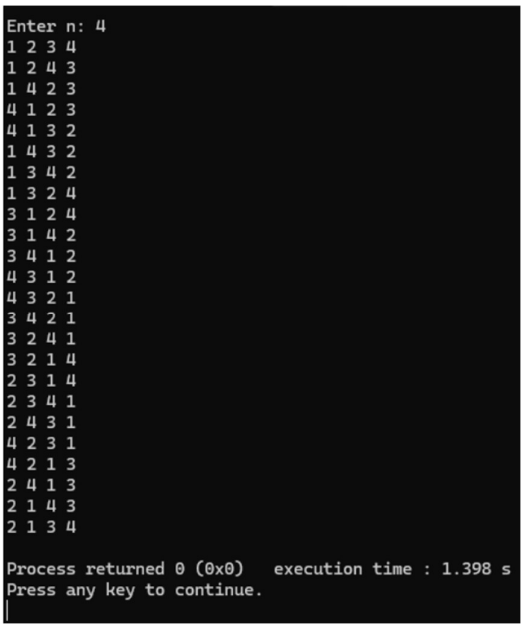
```
#include <stdio.h>
#define l -1
#define r 1
void printPerm(int perm[], int n)
{
   for(int i = 0; i < n; i++)
      printf("%d ", perm[i]);
   printf("\n");
}
int getMobile(int perm[], int dir[], int n)
{
   int mobile = 0;
   for(int i = 0; i < n; i++)
   {
      if(dir[perm[i]-1] == l && i != 0)
      {
         if(perm[i] > perm[i-1] && perm[i] > mobile)
            mobile = perm[i];
      }
      if(dir[perm[i]-1] == r && i != n-1)
      {
         if(perm[i] > perm[i+1] && perm[i] > mobile)
            mobile = perm[i];
      }
   }
   return mobile;
}
```

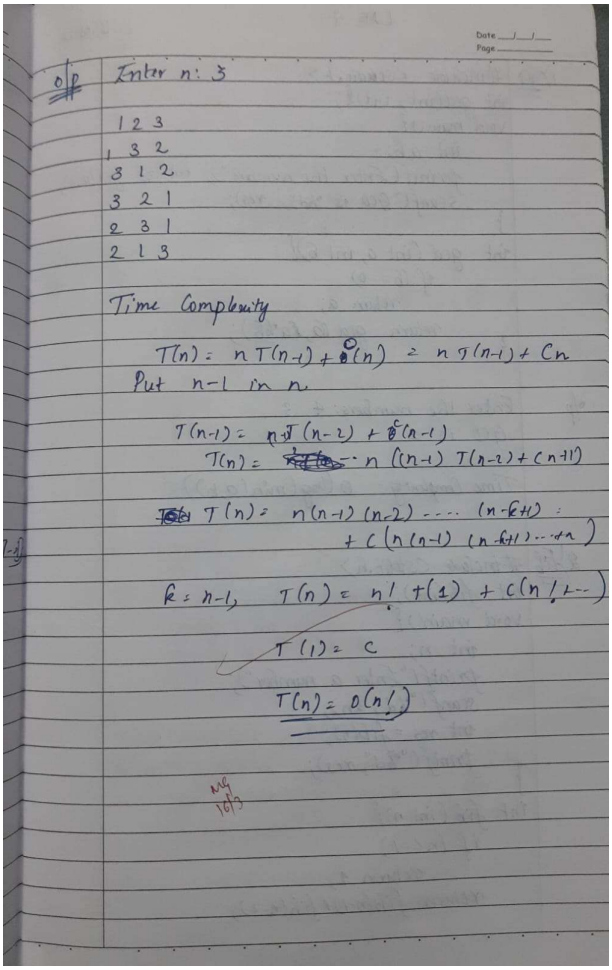
```
int search(int perm[], int n, int mobile)
{
   for(int i = 0; i < n; i++)
      if(perm[i] == mobile)
         return i;
}
void johnsonTrotter(int n)
{
   int perm[n], dir[n];
   for(int i = 0; i < n; i++)
   {
      perm[i] = i + 1;
      dir[i] = l;
   }
   printPerm(perm, n);
   while(1)
   {
      int mobile = getMobile(perm, dir, n);
      if(mobile == 0)
         break;
      int pos = search(perm, n, mobile);
      if(dir[mobile-1] == l)
      {
         int temp = perm[pos];
         perm[pos] = perm[pos-1];
         perm[pos-1] = temp;       }
      else {
         int temp = perm[pos];
         perm[pos] = perm[pos+1];
```

```
            perm[pos+1] = temp;
        }
        for(int i = 0; i < n; i++)
        {
            if(perm[i] > mobile)
                dir[perm[i]-1] = -dir[perm[i]-1];   }
        printPerm(perm, n);    }
}
int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}
```

Output:







Enter n: 3

1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Time Complexity

$T(n) = nT(n-1) + O(n) = nT(n-1) + Cn$

Put $n-1$ in n.

$T(n-1) = (n-1)T(n-2) + O(n-1)$

$T(n) = n(n-1)T(n-2) + C(n+1)$

$T(n) = n(n-1)(n-2)\cdots(n-k+1) + C(n(n-1)(n-k+1)\cdots+n)$

$k = n-1, \quad T(n) = n!\,T(1) + C(n!+\cdots)$

$T(1) = C$

$T(n) = O(n!)$

Lab program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

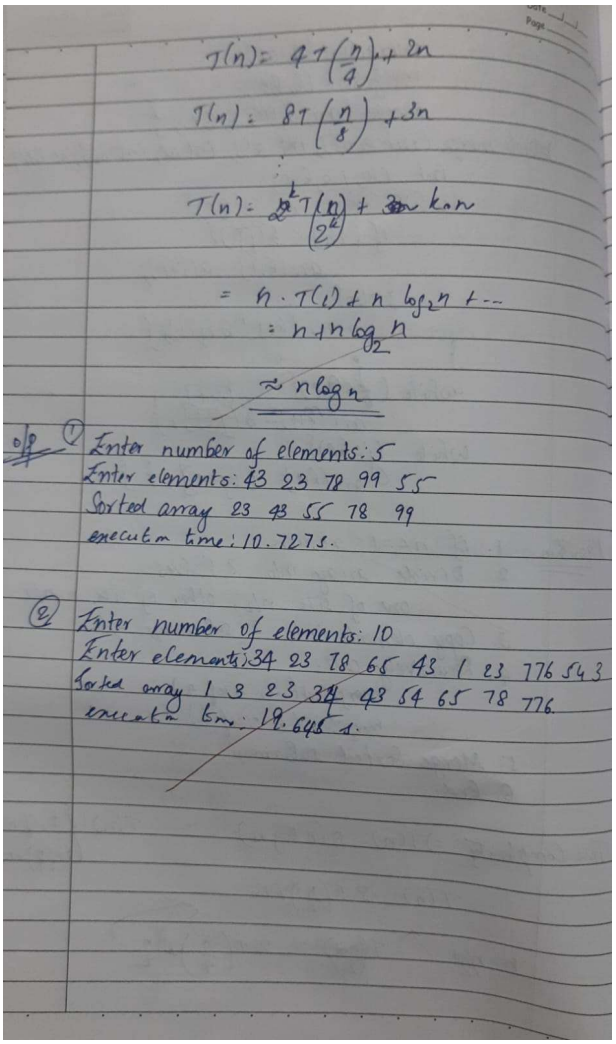
```
#include<stdio.h>
void mergeSort(int arr[], int n);
void merge(int arr[],int a[], int al, int b[], int bl);
void main(){
   int n;
   printf("Enter number of elements:");
   scanf("%d",&n);
   int arr[n];
   printf("Enter elements");
   for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
   }
   mergeSort(arr,n);
   printf("Sorted array");
   for(int i=0;i<n;i++){
      printf("%d ",arr[i]);
   }
}
void mergeSort(int arr[], int n){
   if(n==1) return;
   int al = n/2;
   int bl = n-al;
   int a[al];
   int b[bl];
   for(int i=0;i<al;i++){
      a[i] = arr[i];
```

```
    }
    for(int i=0;i<bl;i++){
        b[i] = arr[al+i];
    }
    mergeSort(a,al);
    mergeSort(b,bl);
    merge(arr,a,al,b,bl);
}
void merge(int arr[],int a[], int al, int b[], int bl){
    int i=0;
    int j=0;
    int k=0;
    while(i<al && j<bl){
        if(a[i]<b[j]){
            arr[k++] = a[i++];
        }
        else{
            arr[k++] = b[j++];
        }
    }
    while(i<al){
        arr[k++] = a[i++];
    }
    while(j<bl){
        arr[k++] = b[j++];
    }
}
```

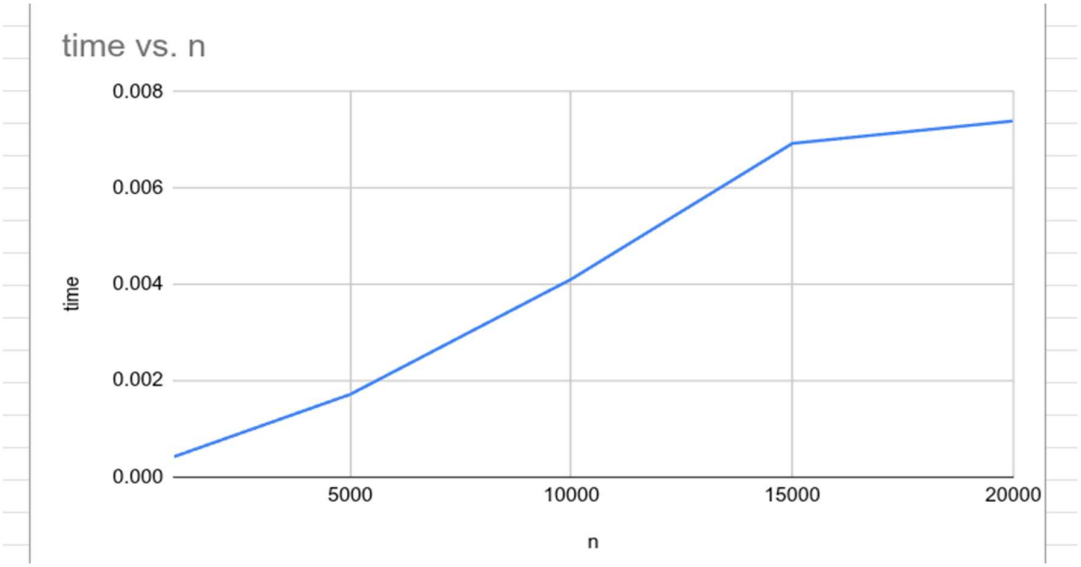
Output:

```
C:\Users\BMSCECSE-L3-66\De
Enter number of elements:5
Enter elements
43 23 78 99 55
Sorted array23 43 55 78 99
Process returned 5 (0x5)   execution time : 42.398 s
Press any key to continue.
```

```
Enter number of elements:10
Enter elements
34 23 78 65 43 1 23 776 54 3
Sorted array1 3 23 23 34 43 54 65 78 776
Process returned 10 (0xA)   execution time : 25.024 s
Press any key to continue.
```



Graph:

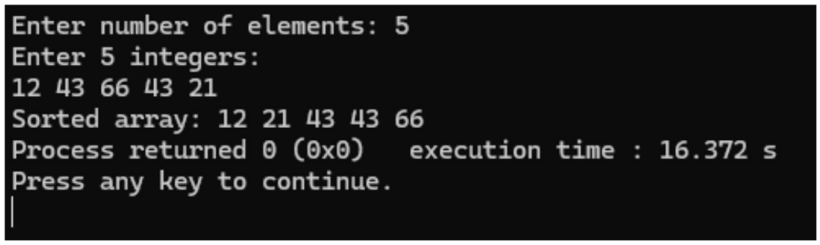

time vs. n
0.008
0.006
0.004
0.002
0.000
time
5000
10000
15000
20000
n

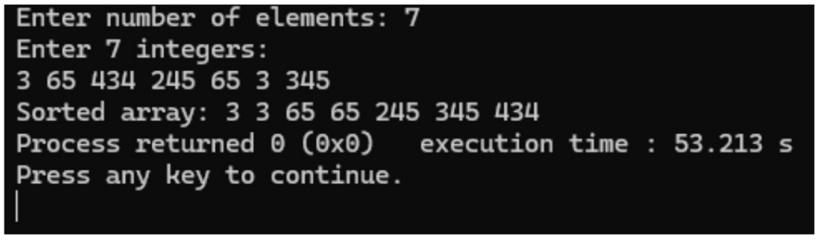
Lab program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken

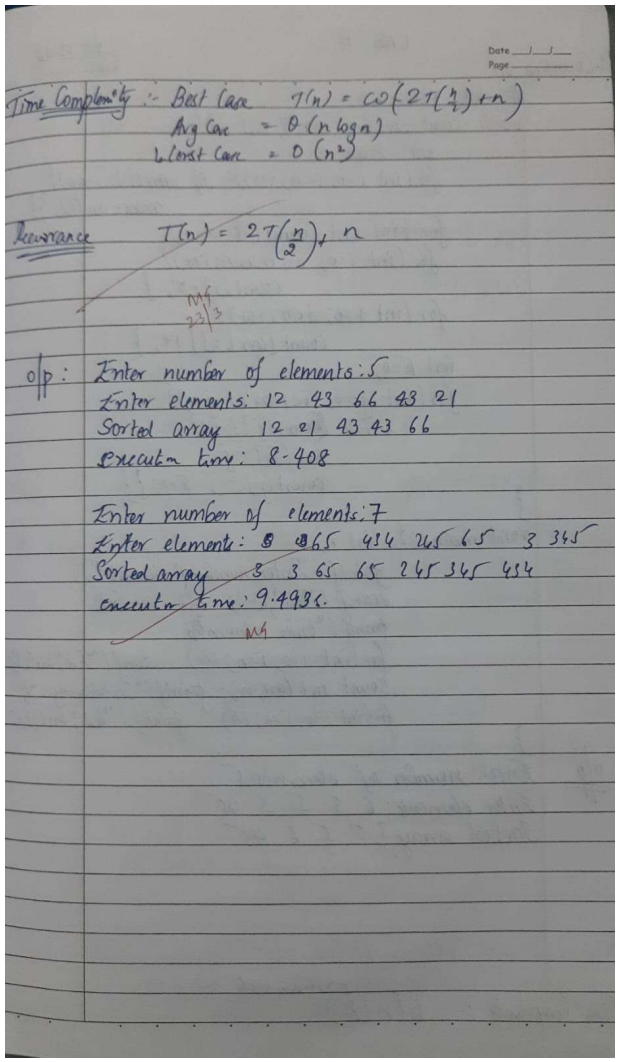
```
#include <stdio.h>
void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}
int partition(int arr[], int low, int high) {
   int pivot = arr[high];  // choose last element as pivot
   int i = (low - 1);
   for (int j = low; j < high; j++) {
      if (arr[j] <= pivot) {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return (i + 1);}
void quickSort(int arr[], int low, int high) {
   if (low < high) {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);  }
}
int main() {
   int n;
   printf("Enter number of elements: ");
   scanf("%d", &n);
```

```
    int arr[n];
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);  }
    quickSort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);    }
    return 0;
}
```

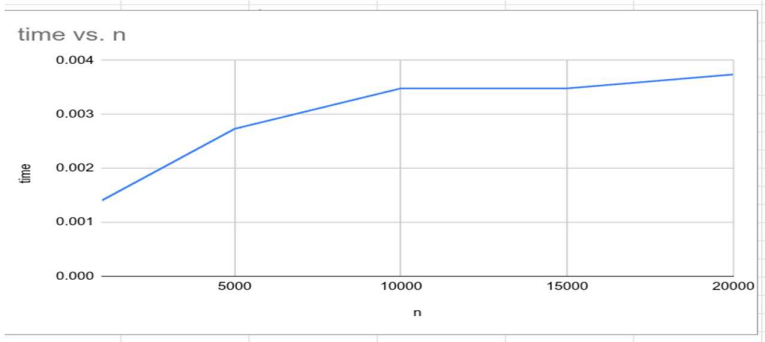
Output:






Time Complexity :- Best Case T(n) = cn(2T(n/2)+n)
Avg Case = θ(n log n)
Worst Case = O(n²)
Recurrance T(n) = 2T(n/2) + n
o/p : Enter number of elements: 5
Enter elements: 12 43 66 43 21
Sorted array 12 21 43 43 66
Execution time: 8.408
Enter number of elements: 7
Enter elements: 3 65 434 245 65 3 345
Sorted array 3 3 65 65 245 345 434
execution time: 9.493s.

Graph:


time vs. n
time
0.004
0.003
0.002
0.001
0.000
5000
10000
15000
20000
n

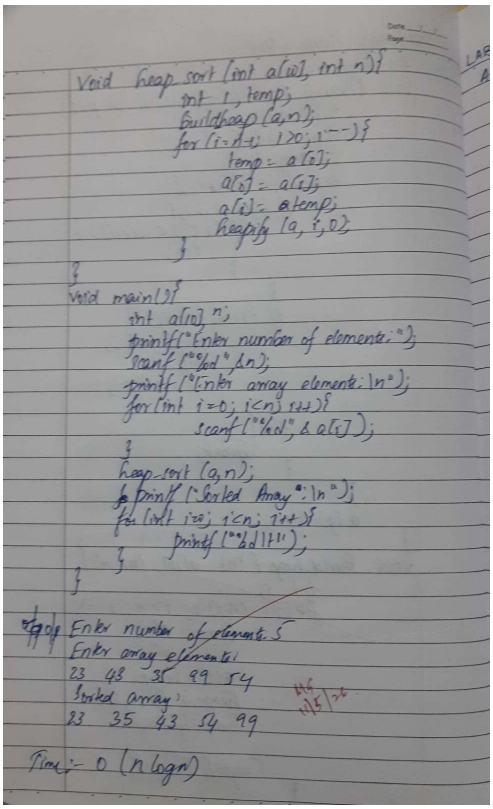
Lab program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
void heapify(int a[10],int n,int p){
   int c,item;
   item=a[p];
   c=2*p+1;
   while(c<=n-1){
      if(c+1<=n-1){
         if (a[c]<a[c+1]){
            c++;
         }}
         if(item<a[c]){
            a[p]=a[c];
            p=c;
            c=2*p+1;
         }
         else{
            break;
         }
      }
      a[p]=item;
}
void build_heap(int a[10],int n){
   int i;
   for(i=(n-1)/2;i>=0;i--){
      heapify(a,n,i);
   }
}
```

```
void heap_sort(int a[10],int n){
   int i,temp;
   build_heap(a,n);
   for(i=n-1;i>0;i--){
      temp=a[0];
      a[0]=a[i];
      a[i]=temp;
      heapify(a,i,0);
   }
}
void main(){
   int a[10],n;
   printf("Enter number of integers:");
   scanf("%d",&n);
   printf("Enter array elements:\n");
   for(int i=0;i<n;i++){
      scanf("%d",&a[i]);
   }
   heap_sort(a,n);
   printf("Sorted array:\n");
   for(int i=0;i<
n;i++){
      printf("%d\t",a[i]);
   }
}
```

Output:

```
C:\Users\BMSCECSE-L3-66\De
Enter number of integers:5
Enter array elements:
23 43 35 99 54
Sorted array:
23      35      43      54      99
Process returned 5 (0x5)   execution time : 15.087 s
Press any key to continue.
```

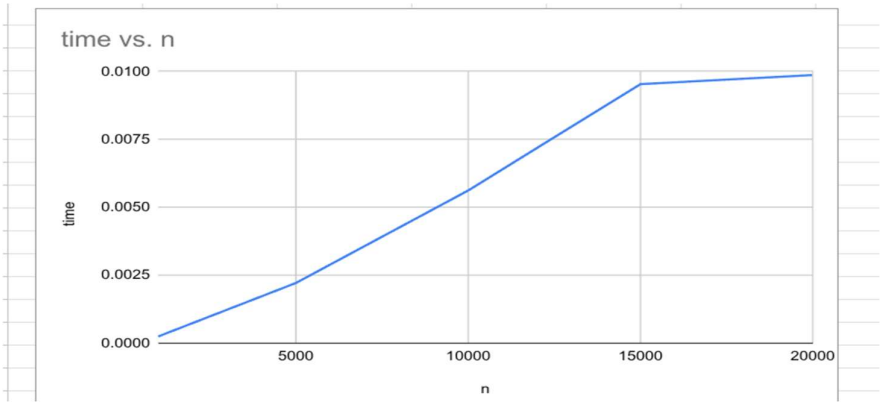


```
void heap_sort(int a[], int n){
    int i, temp;
    buildheap(a,n);
    for(i=n-1; i>0; i--){
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
}
void main(){
    int a[10], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements:\n");
    for(int i=0; i<n; i++){
        scanf("%d", &a[i]);
    }
    heap_sort(a,n);
    printf("Sorted Array:\n");
    for(int i=0; i<n; i++){
        printf("%d\t", a[i]);
    }
}
```

Enter number of elements 5
Enter array elements:
23 43 35 99 54
Sorted array:
23 35 43 54 99

Time :- O(n log n)

Graph:

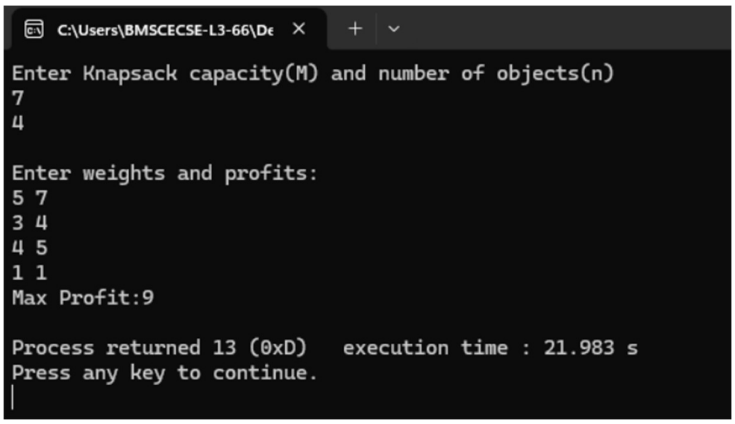


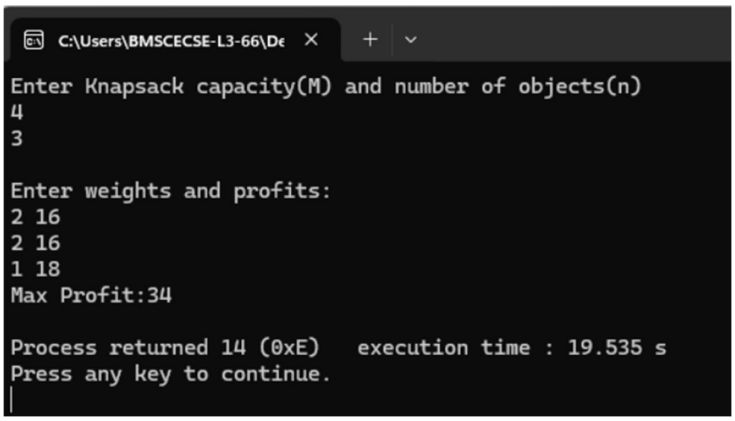
Lab program 6: Implement 0/1 Knapsack problem using dynamic programming.

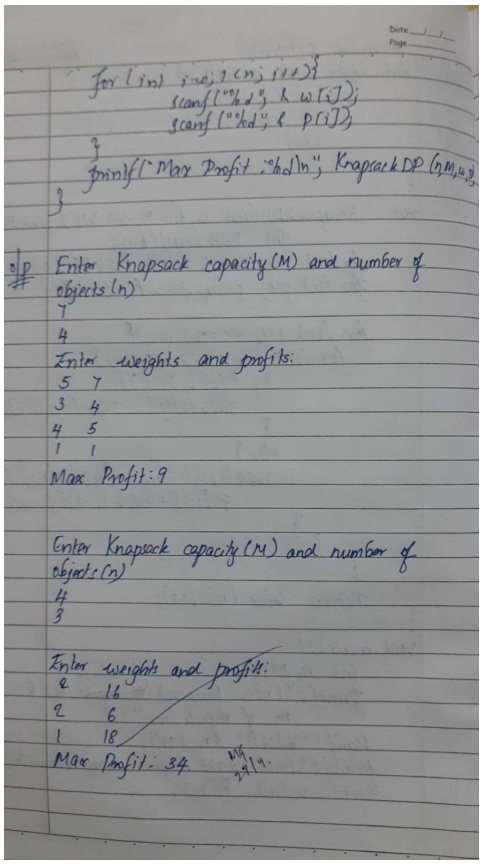
```
#include <stdio.h>
int max(int a, int b){
   if (a>b){
      return a;    }
   else{
      return b;}
}
int KnapsackDP(int n,int M,int W[],int P[]){
   int Table[n+1][M+1];
   for(int i=0;i<=n;i++) Table[i][0]=0;
   for(int j=0;j<=M;j++) Table[0][j]=0;
   for(int i=1;i<=n;i++){
      for(int j=1;j<=M;j++){
         if(j<W[i-1]){
            Table[i][j]=Table[i-1][j];
         }
         else{
            Table[i][j]=max(Table[i-1][j],P[i-1]+Table[i-1][j-W[i-1]]);
         }
      }
   }
   return Table[n][M];
}
void main(){
   int n,M;
   printf("Enter Knapsack capacity(M) and number of objects(n)");
   scanf("%d%d",&M,&n);
```

```
    printf("\nEnter weights and profits:\n");
    int w[n],P[n];
    for(int i=0;i<n;i++){
        scanf("%d%d",&w[i],&P[i]);
    }
    printf("Max Profit:%d\n",KnapsackDP(n,M,w,P));
}
```

Output:







```
for (int i=1; i<=n; i++){
    scanf("%d", &w[i]);
    scanf("%d", &p[i]);
}
printf("Max Profit :%d\n", KnapsackDP(n,M,w,p));
}
```

o/p

Enter Knapsack capacity (M) and number of objects (n)
7
4
Enter weights and profits:
5 7
3 4
4 5
1 1
Max Profit: 9

Enter Knapsack capacity (M) and number of objects (n)
4
3

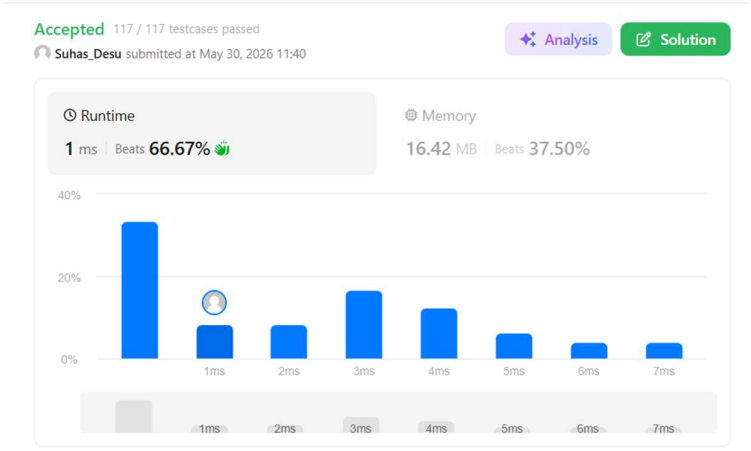
Enter weights and profits:
2 16
2 6
1 18
Max Profit: 34

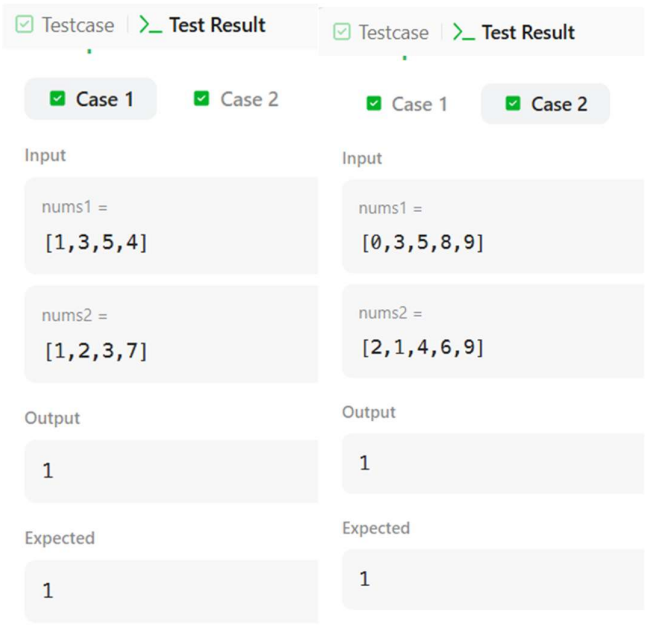
Leet Code exercise on Knapsack problem using dynamic programming.

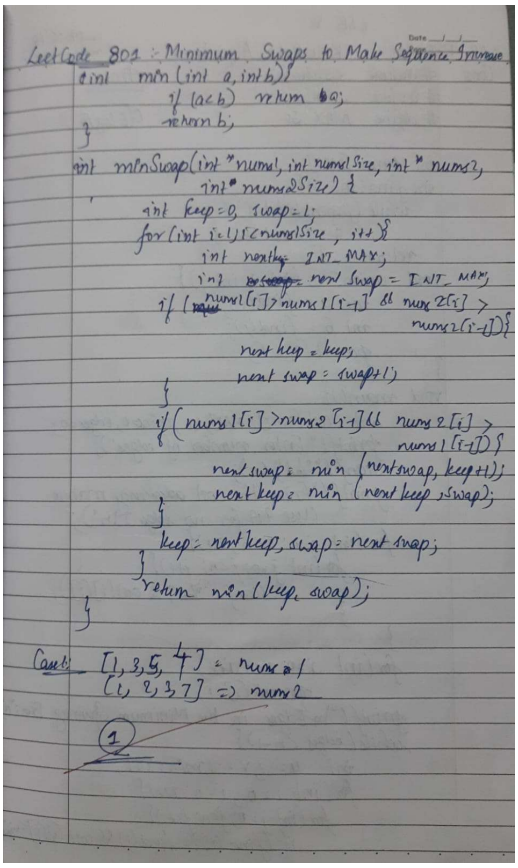
Minimum Swaps to Make Sequences Increasing (Leet Code 801)

```
int min(int a,int b){
    if(a<b) return a;
    return b;
}
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep=0,swap=1;
    for(int i=1;i<nums1Size;i++){
        int next_keep= INT_MAX;
        int next_swap=INT_MAX;
        if(nums1[i]>nums1[i-1] && nums2[i]>nums2[i-1]){
            next_keep=keep;
            next_swap=swap+1;
        }
        if(nums1[i]>nums2[i-1] && nums2[i]>nums1[i-1]){
            next_swap=min(next_swap,keep+1);
            next_keep=min(next_keep,swap);
        }
        keep=next_keep;
        swap=next_swap;
    }
    return min(keep,swap);
}
```

Output:


Accepted 117 / 117 testcases passed
Suhas_Desu submitted at May 30, 2026 11:40
Analysis
Solution
Runtime
1 ms | Beats 66.67%
Memory
16.42 MB | Beats 37.50%
40%
20%
0%
1ms
2ms
3ms
4ms
5ms
6ms
7ms


Testcase
Test Result
Case 1
Case 2
Input
nums1 =
[1,3,5,4]
nums2 =
[1,2,3,7]
Output
1
Expected
1
Testcase
Test Result
Case 1
Case 2
Input
nums1 =
[0,3,5,8,9]
nums2 =
[2,1,4,6,9]
Output
1
Expected
1


LeetCode 801 : Minimum Swaps to Make Sequences Increasing

```
int min(int a, int b){
    if(a<b) return a;
    return b;
}
int minSwap(int *nums1, int nums1Size, int *nums2,
    int* nums2Size) {
    int keep=0, swap=1;
    for(int i=1;i<nums1Size ; i++){
        int nextkeep= INT_MAX;
        int nextswap = INT_MAX;
        if(nums1[i]>nums1[i-1] && nums2[i]>
            nums2[i-1]){
            nextkeep = keep;
            nextswap = swap+1;
        }
        if(nums1[i]>nums2[i-1] && nums2[i]>
            nums1[i-1]){
            nextswap= min(nextswap, keep+1);
            nextkeep= min(nextkeep, swap);
        }
        keep= nextkeep, swap= nextswap;
    }
    return min(keep, swap);
}
```

Case: [1,3,5,4] = nums1
[1,2,3,7] => nums2

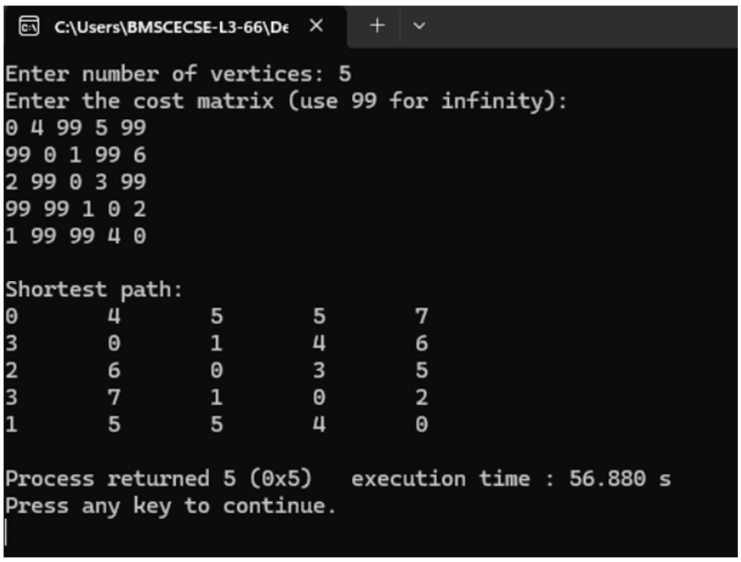
(1)

Lab program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define INF 99
void floyds(int n, int cost[10][10], int A[10][10]) {
   int i, j, k;
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         A[i][j] = cost[i][j];
      }
   }
   for (k = 1; k <= n; k++) {
      for (i = 1; i <= n; i++) {
         for (j = 1; j <= n; j++) {
            if (A[i][k] + A[k][j] < A[i][j]) {
               A[i][j] = A[i][k] + A[k][j];
            }
         }
      }
   }
}
void main() {
   int n, i, j;
   int cost[10][10], A[10][10];
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter the cost matrix (use 99 for infinity):\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
```

```
            scanf("%d", &cost[i][j]);
        }
    }
    floyds(n, cost, A);
    printf("\nShortest path:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (A[i][j] == INF)
                printf("INF\t");
            else
                printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
}
```

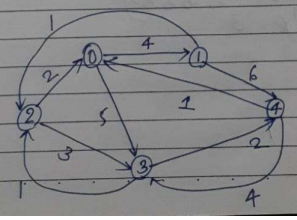
Output:



```
floyds (n, cost, A);
printf ("\n Shortest Path: \n");
for (int i = 1; i <= n; i++){
    for (j = 1; j <= n; j++){
        if (A[i][j] == INF)
            printf ("INF \t");
        else
            printf ("%d \t", A[i][j]);
    }
    printf ("\n");
}
}
```

o/p

```
Enter number of vertices: 5
Enter the cost matrix (use 99 for infinity):
0  4  99  5  99
99 0  1   99 6
2  99 0   3  99
99 99 1   0  2
1  99 99  4  0
Shortest Path:
0   4   5   5   7
3   0   1   4   6
2   6   0   3   5
3   7   1   0   2
1   5   5   4   0
```

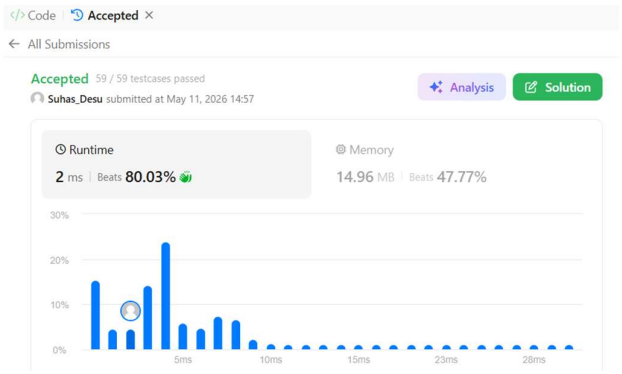


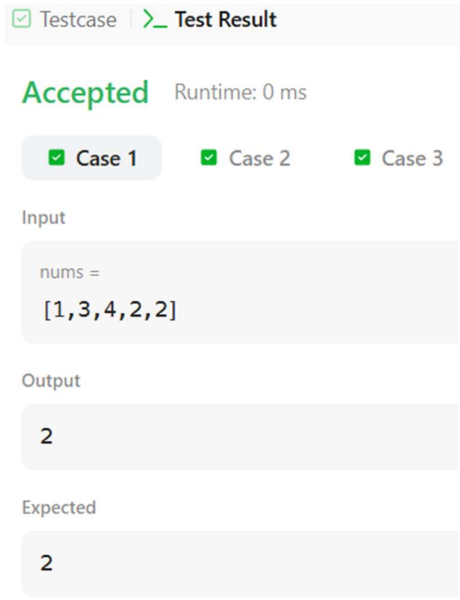
Leet Code exercise on Floyd's algorithm.

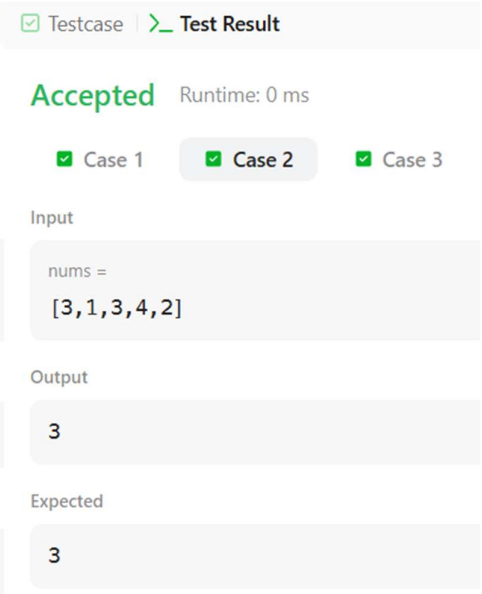
Find the Duplicate Number (Leet Code 287)

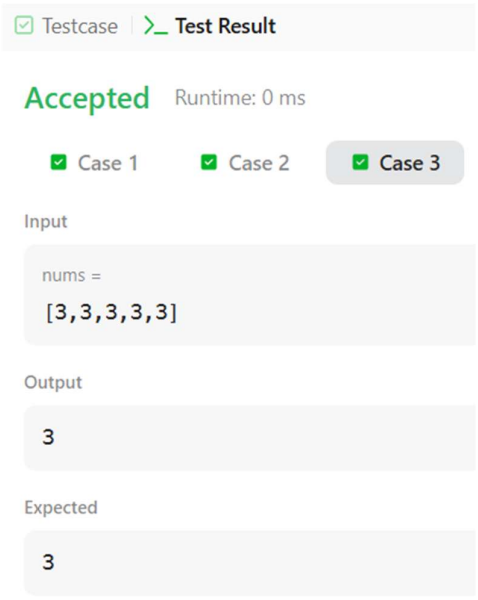
```
int findDuplicate(int* nums, int numsSize) {
    int slow = nums[0];
    int fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}
```

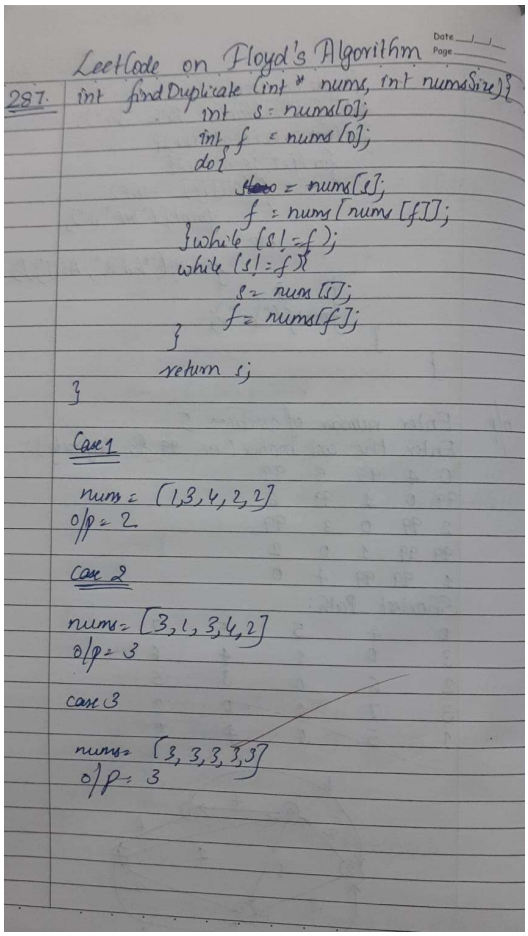
Output:




Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,4,2,2]
Output
2
Expected
2


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[3,1,3,4,2]
Output
3
Expected
3


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[3,3,3,3,3]
Output
3
Expected
3



LeetCode on Floyd's Algorithm

287. int findDuplicate(int* nums, int numsSize){
    int s = nums[0];
    int f = nums[0];
    do{
        s = nums[s];
        f = nums[nums[f]];
    }while(s!=f);
    while(s!=f){
        s = nums[s];
        f = nums[f];
    }
    return s;
}

Case 1

nums = [1,3,4,2,2]
o/p = 2

Case 2

nums = [3,1,3,4,2]
o/p = 3

case 3

nums = [3,3,3,3,3]
o/p = 3

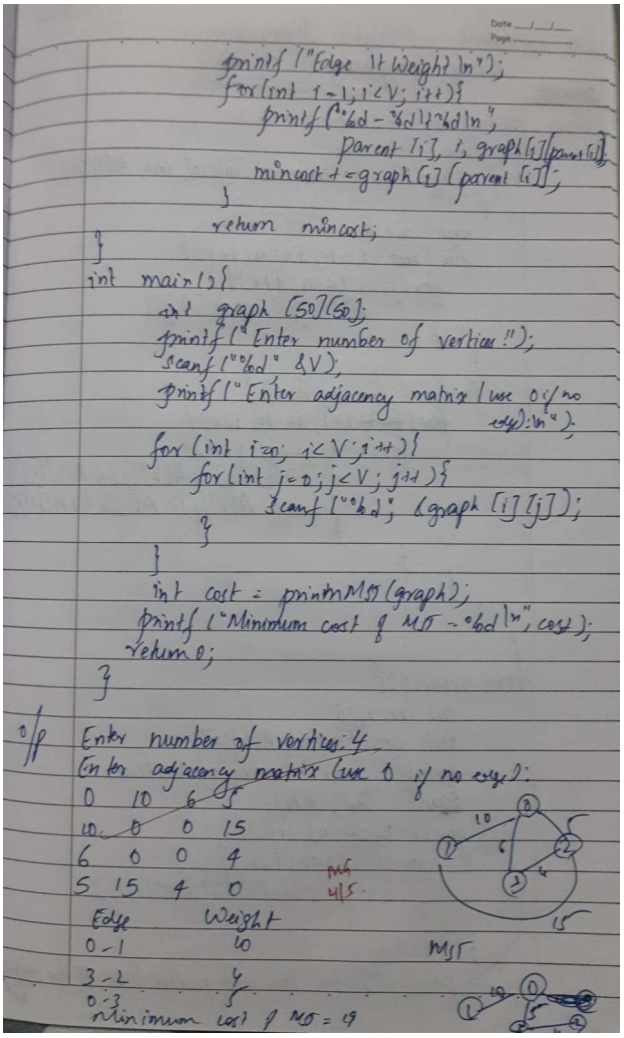
Lab program 8 (a): Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#define INF 999999
int V;
int minKey(int key[], int mstSet[]) {
   int min = INF, min_index = -1;
   for (int v = 0; v < V; v++) {
      if (mstSet[v] == 0 && key[v] < min) {
         min = key[v];
         min_index = v;
      }
   }
   return min_index;
}
int primMST(int graph[50][50]) {
   int parent[50];
   int key[50];
   int mstSet[50];
   int mincost = 0;
   for (int i = 0; i < V; i++) {
      key[i] = INF;
      mstSet[i] = 0;
   }
   key[0] = 0;
   parent[0] = -1;
   for (int count = 0; count < V - 1; count++) {
      int u = minKey(key, mstSet);
      mstSet[u] = 1;
      for (int v = 0; v < V; v++) {
```

```
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        mincost += graph[i][parent[i]];
    }
    return mincost;
}
int main() {
    int graph[50][50];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix (use 0 if no edge):\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int cost = primMST(graph);
    printf("Minimum cost of MST = %d\n", cost);
    return 0;
}
```

Output:

```
C:\Users\BMSCECSE-L3-66\De
Enter number of vertices: 4
Enter adjacency matrix (use 0 if no edge):
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edge    Weight
0 - 1   10
3 - 2   4
0 - 3   5
Minimum cost of MST = 19

Process returned 0 (0x0)   execution time : 23.567 s
Press any key to continue.
```



Lab program 8 (b): Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

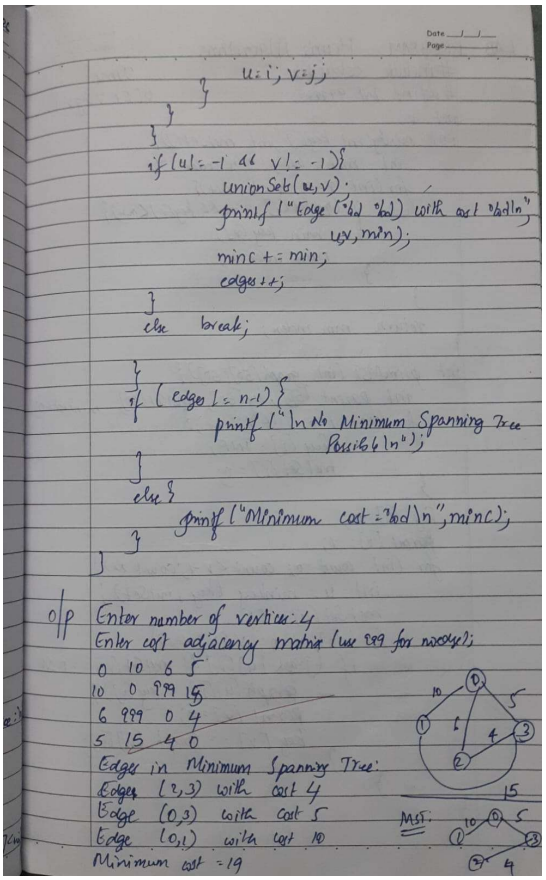
```
#include <stdio.h>
#include <stdlib.h>
#define MAX 30
int parent[MAX];
int find(int i) {
   while (parent[i] != i)
      i = parent[i];
   return i;}
void unionSets(int i, int j) {
   int a = find(i);
   int b = find(j);
   parent[a] = b;
}
int main() {
   int cost[MAX][MAX];
   int n;
   int mincost = 0;
   int edges = 0;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter the cost adjacency matrix (use 999 for no edge):\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
      }
   }
   for (int i = 0; i < n; i++)
```

```
    parent[i] = i;
  printf("\nEdges in the Minimum Spanning Tree:\n");
  while (edges < n - 1) {
    int u = -1, v = -1;
    int min = 999;
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        if (find(i) != find(j) && cost[i][j] < min) {
          min = cost[i][j];
          u = i;
          v = j;
        }
      }
    }
    if (u != -1 && v != -1) {
      unionSets(u, v);
      printf("Edge (%d, %d) with cost %d\n", u, v, min);
      mincost += min;
      edges++;
    } else {
      break;
    }
  }
  if (edges != n - 1) {
    printf("\nNo Minimum Spanning Tree Possible\n");
  } else {
    printf("Minimum cost = %d\n", mincost);
  }
```

```
    return 0;
}
```

Output:

```
C:\Users\BMSCECSE-L3-66\De
Enter number of vertices: 4
Enter the cost adjacency matrix (use 999 for no edge):
0 10 6 5
10 0 999 5
6 999 0 54
5 15 4 0

Edges in the Minimum Spanning Tree:
Edge (3, 2) with cost 4
Edge (0, 3) with cost 5
Edge (1, 3) with cost 5
Minimum cost = 14

Process returned 0 (0x0)   execution time : 28.398 s
Press any key to continue.
```

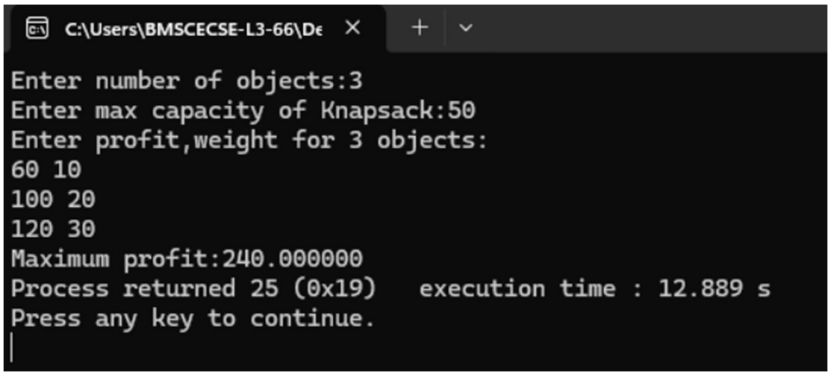


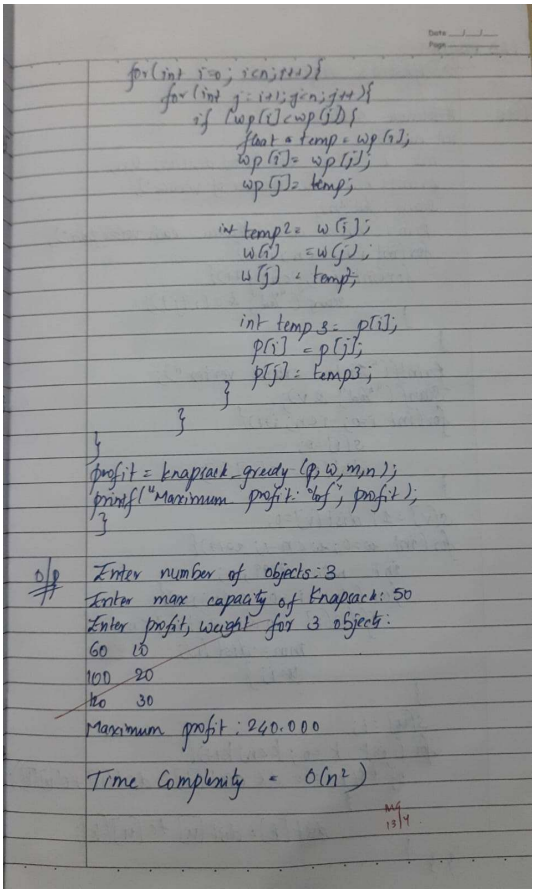
Lab program 9: Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>
float knapsack_greedy(int p[],int w[],int m,int n){
    float x[n],max_profit=0.0;
    for(int i=0;i<n;i++){
        x[i]=0.0;}
    int rem_cap=m;
    int i=0;
    while (rem_cap>0){
        if (w[i]>rem_cap) break;
        x[i]=1.0;
        rem_cap=rem_cap-w[i];
        i++;}
    if (rem_cap!=0 && i<n) x[i]=(float)rem_cap/w[i];
    for(int i=0;i<n;i++){
        max_profit=max_profit+(x[i]*p[i]);}
    return max_profit;
}
void main(){
    int n,m;
    printf("Enter number of objects:");
    scanf("%d",&n);
    printf("Enter max capacity of Knapsack:");
    scanf("%d",&m);
    int p[n],w[n];
    float wp[n],profit;
    printf("Enter profit,weight for %d objects:\n",n);
    for(int i=0;i<n;i++){
        scanf("%d%d",&p[i],&w[i]);
```

```
        wp[i]=(float)p[i]/w[i];
    }
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(wp[i]<wp[j]){
                float temp=wp[i];
                wp[i]=wp[j];
                wp[j]=temp;
                int temp2=w[i];
                w[i]=w[j];
                w[j]=temp2;
                int temp3=p[i];
                p[i]=p[j];
                p[j]=temp3;
                }
        }
    }
    profit=knapsack_greedy(p,w,m,n);
    printf("Maximum profit:%f",profit);
}
```

Output:





```
for(int i=0; i<n;i++){
    for(int j=i+1;j<n;j++){
        if(wp[i]<wp[j]){
            float temp = wp[i];
            wp[i]= wp[j];
            wp[j]= temp;

            int temp2= w[i];
            w[i] =w[j];
            w[j] = temp2;

            int temp3= p[i];
            p[i] = p[j];
            p[j]= temp3;
        }
    }
}
profit = knapsack_greedy (p, w, m, n);
printf("Maximum profit: %f", profit);
}
```

O/P

```
Enter number of objects: 3
Enter max capacity of knapsack: 50
Enter profit, weight for 3 objects:
60 10
100 20
120 30
Maximum profit: 240.000
```

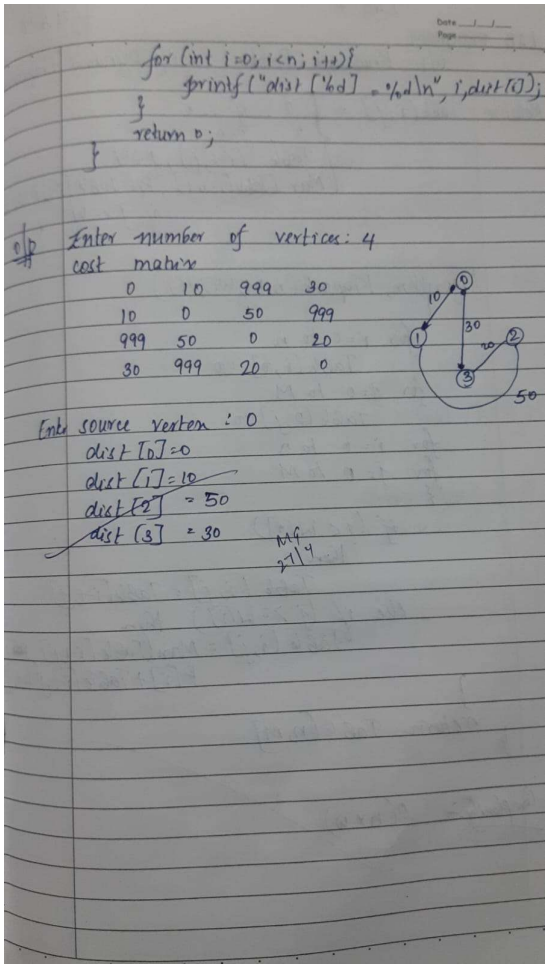
Time Complexity = $O(n^2)$

Lab program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
void main(){
   int n,v,c[100][100],s[100],dist[100];
   printf("Enter number of vertices:");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
         for(int j=0;j<n;j++){
            scanf("%d",&c[i][j]);
         }
   }
   printf("Enter source vertex");
   scanf("%d",&v);
   for(int i=0;i<n;i++){
      s[i]=0;
      dist[i]=c[v][i];
   }
   s[v]=1;
   dist[v]=0;
   for(int w=0;w<n-1;w++){
      int min=999,u;
      for(int i=0;i<n;i++){
         if(s[i]==0 && dist[i]<min){
            min=dist[i];
            u=i;
         }
      }
      s[u]=1;
      for(int k=0;k<n;k++){
         if(s[k]==0&& dist[k]>dist[u]+c[u][k]){
            dist[k]=dist[u]+c[u][k];
         }
      }
   }
   for(int i=0;i<n;i++){
      printf("dist[%d]=%d\n",i,dist[i]);
   }
}
```

Output:

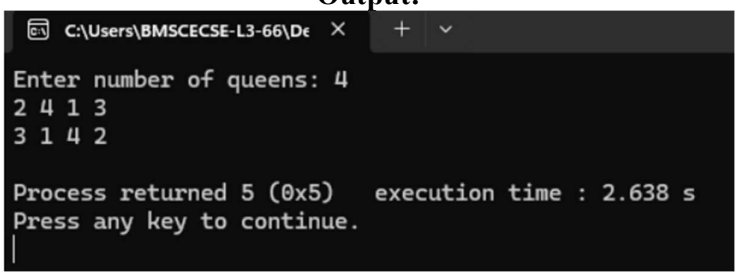
```
Enter number of vertices:4
0 10 999 30
10 0 50 999
999 50 0 20
30 999 20 0
Enter source vertex0
dist[0]=0
dist[1]=10
dist[2]=50
dist[3]=30

Process returned 4 (0x4)   execution time : 71.544 s
Press any key to continue.
```

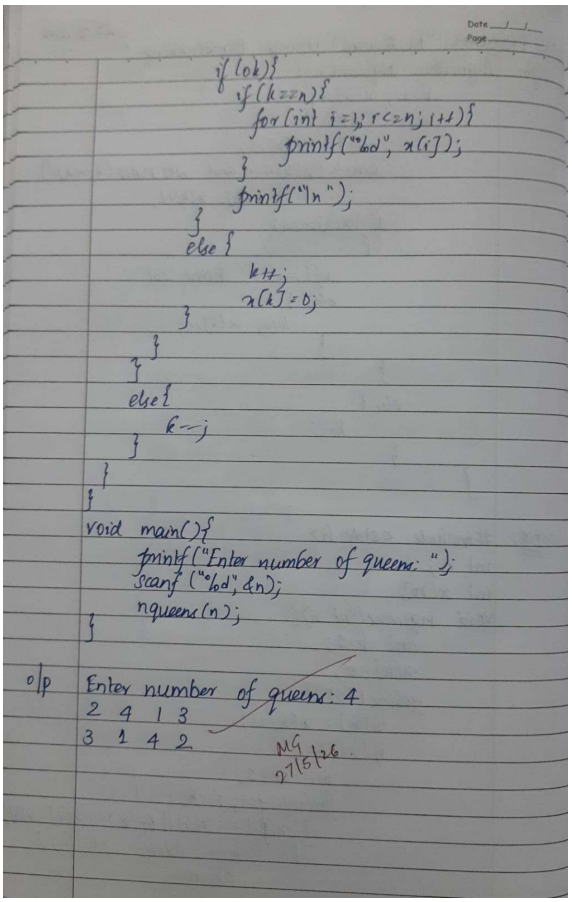


Lab program 11: Implement "N-Queens Problem" using Backtracking.

```
#include <stdio.h>
int n;
int x[20];
void nqueens(int n) {
   int k = 1;
   x[k] = 0;
   while (k > 0) {
      x[k] = x[k] + 1;
      if (x[k] <= n) {
         int ok = 1;
         for (int j = 1; j < k; j++) {
            if (x[j] == x[k] || (j - k) == (x[j] - x[k]) || (j - k) == (x[k] - x[j])) {
               ok = 0;
               break;
            }
         }
         if (ok) {
            if (k == n) {
               for (int i = 1; i <= n; i++) {
                  printf("%d ", x[i]);
               }
               printf("\n");
            } else {
               k++;
               x[k] = 0;
            }
         }
      } else {
         k--;
      }
   }
}
void main() {
   printf("Enter number of queens: ");
   scanf("%d", &n);
   nqueens(n);
}
```

Output:





```
        if(ok){
          if(k==n){
            for(int i=1; i<=n; i++){
              printf("%d", x[i]);
            }
            printf("\n");
          }
          else{
            k++;
            x[k]=0;
          }
        }
      }
      else{
        k--;
      }
    }
  }
  void main(){
    printf("Enter number of queens: ");
    scanf("%d", &n);
    nqueens(n);
  }

o/p  Enter number of queens: 4
     2 4 1 3
     3 1 4 2
```

MG 27/6/26